Oracle-Anchored LVR Recapture with Dutch Auction Repricing

Joel Kahil
joel@nethermind.io

Mitchell Marfinetz
mitchell.marfinetz@nethermind.io

May 2026

Abstract

Loss-versus-rebalancing is the adverse-selection cost AMM LPs pay when a pool quotes stale prices against a faster reference market. This paper studies a Uniswap v4 hook that detects stale pool prices from the pool-oracle gap, charges informed stale-price trades with the square-root fee formula, and routes repricing through a Dutch auction. The empirical test uses the October 2025 block-by-block replay for WETH/USDC, WBTC/USDC, LINK/WETH, and UNI/WETH. Across 1,296 pool-parameter rows and 40,176 window-parameter rows, the recommended set uses a 10 bps trigger, 5 bps base fee, 10 bps starting solver concession, 0.5 bps/sec concession growth, and 2500 bps max fee. In this controlled replay without gas costs or competitive routing, it beats fixed-fee V3 on all four pools, clears every modeled auction under that set, and leaves solvers 10 bps of gross stale-LVR value, about 0.1% of each stale-value opportunity. A secondary observed-flow replay covers 54 windows across seven pools and finds that the hook-based auction rule improves LP net gain versus the broader all-stale auction rule in 28 windows while worsening none.

1 Introduction

Uniswap pools quote from on-chain state. When an external reference market moves first, informed traders can rebalance the stale pool and extract value from LPs. These informed repricing trades are the AMM setting's toxic flow: order flow that is negative for LPs because it trades against stale quotes and creates loss before fees. The loss-versus-rebalancing literature gives this cost a precise accounting interpretation: LPs lose relative to a continuously rebalanced inventory benchmark when they are forced to trade at stale prices.

The design question here is narrower than a general AMM redesign. We ask whether a Uniswap v4 hook can identify stale prices from a pool-oracle gap, charge informed trades in the stale direction according to an exact fee formula, constrain newly admitted concentrated liquidity, and use an auctioned repricing path to preserve execution instead of simply deterring trade. The answer matters because a hook-only defense can look good in LP accounting while leaving the pool stale or admitting fragile marginal liquidity. A usable design must do three things at once: redirect stale-price value to LPs, bound the curvature risk LPs add ex ante, and keep repricing active.

The empirical target is the stale pool state itself. Triggering on absolute price differences or oracle movement is not portable across tokens. The trigger must be the current pool-oracle stale gap, expressed in basis points and shared by every selection rule. The backtest therefore uses one auction trigger, a parameter grid anchored on real fee tiers, and four-pool consistency as the main check.

2 Related Work and Positioning

This paper follows the LVR framing of automated market maker adverse selection, but it is not a continuous-time model. It turns the one-step stale-price accounting formula into a hook-level execution rule and then tests that rule on frozen Uniswap data. The closest comparison is auction-managed AMM work, but the auction here is deliberately narrower. It does not auction pool governance or general order flow. It sells a short-lived repricing right only after a stale pool price is detected.

The design also differs from generic dynamic-fee AMMs. The fee is not a volatility heuristic or volume response. For informed trades against stale prices, the deterministic hook fee is the original square-root premium formula, capped by a configured max fee. The Dutch auction sits around that fee floor as the repricing path. Its job is to determine whether a solver can reprice the pool while leaving LPs better protected than a fixed-fee V3 pool.

3 Glossary

The main object is a **Stale position**: a pool state satisfying

$$\texttt{stale_gap_bps_before} \geq \texttt{trigger_gap_bps}.$$

The **Stale gap** is unsigned for eligibility:

$$\texttt{stale_gap_bps_before} = |\log(\texttt{oracle_mid}/\texttt{pool_mid})| \times 10{,}000,$$

and is computed through the repository's existing `gap_bps()` helper. Direction is stored separately as `stale_gap_sign = sign(oracle_mid - pool_mid)`. Positive sign means the oracle is above the pool, so the pool is stale low.

The **Auction trigger** is the shared helper `is_auction_eligible(state, trigger_gap_bps)`. The selection rules all use this trigger and differ only in how they rank parameter sets. An **Informed repricing trade** is the modeled trade that moves a stale pool toward the faster reference price. This is the paper's **toxic flow**: flow that is adverse to LPs because it trades against stale quotes and creates LP loss before fees. **LP loss** is `gross_lvr` − `fee_revenue`. **LP net** is `fee_revenue` − `gross_lvr`. The **auction-beats-hook branch** counts an auction only when it improves LP accounting relative to the base hook fee.

Recapture is

$$\texttt{recapture_pct} = \frac{\texttt{lp_fee_revenue_quote}}{\texttt{gross_lvr_quote}} \times 100.$$

Cross-pool consistency is measured as the number of pools where a parameter set beats the matching fixed-fee V3 baseline while maintaining auction clear rate at or above the working 0.5 threshold. The **24-hour window check** is not a protocol policy in this paper. It is a temporal replay diagnostic used to check whether the monthly result is concentrated in a small part of October. A **historical stress window** is a replay slice selected from periods with larger oracle-pool dislocation, oracle disagreement, high activity, or known replay edge cases. It is still historical data, not a Brownian-motion synthetic path. The purpose is to test stability and replay reliability under harder market slices, not to define an auction trigger.

4 Assumptions

The empirical model is intentionally narrow. It assumes a **Single rational arbitrageur** with one-block oracle foresight. The **Reference oracle** is Chainlink primary, with Pyth, Binance, and

deep-pool references reserved for sensitivity checks. There are **No gas costs** in the simulation. The repository contains exact V3 replay, informed/uninformed flow labels, and historical swap replay through fee curves; informed means toxic to LPs under the stale-price accounting above, while uninformed means ordinary flow in this model. The reported auction grid uses those window inputs to evaluate **Simulated correction trades** on stale repricing opportunities, rather than re-routing every historical user swap through a competitive multi-venue environment. It has a **Single solver** bidding on the auction, with no competitive solver dynamics. The comparison baseline is **Fixed-fee V3** at the same pool fee tier. The resolved **October 2025 dataset** is the only numerical data source for the reported grid:

`.tmp/month_2025_10/checkpointed_export_combined/windows`.

These assumptions make the result a controlled stale-repricing test rather than a production market simulation. They are strong assumptions, but they are useful because they isolate the main question: when a stale repricing opportunity exists, can the hook-auction design transfer most of the stale-price value to LPs while preserving execution?

5 Fee Formula and Hook Design

The Dutch auction needs a reserve price. That reserve is not an arbitrary fee knob: it comes from the one-step stale-price loss created when a constant-product pool is repriced against a faster reference market. The fee formula below defines the base hook price that the auction must improve upon.

Let the pool mid be P and the reference mid be $P' = Pe^z$, where z is the signed log gap. For a constant-product pool with invariant $xy = L^2$, the pool state aligned with price P is

$$x(P) = \frac{L}{\sqrt{P}}, \qquad y(P) = L\sqrt{P}.$$

For a constant-product repricing from P to P', the informed input needed to move the pool creates stale-price value. The exact fee fraction on that input that recovers the one-step stale loss is

$$f^\star(z) = e^{|z|/2} - 1.$$

The proof is symmetric across directions. If $z > 0$, the oracle is above the pool and the informed trader pays the quote asset into the pool to buy the base asset too cheaply. The quote-asset input and base-asset output are

$$y_{\text{in}} = L\sqrt{P}(e^{z/2} - 1), \qquad x_{\text{out}} = \frac{L}{\sqrt{P}}(1 - e^{-z/2}).$$

The stale-price loss, valued at the new reference price P', is

$$\Pi_{\text{stale}} = P' x_{\text{out}} - y_{\text{in}} = L\sqrt{P}(e^{z/2} - 1)^2.$$

Charging fee fraction f on the informed input gives fee revenue $f y_{\text{in}}$. Setting fee revenue equal to stale-price loss gives

$$f = \frac{L\sqrt{P}(e^{z/2} - 1)^2}{L\sqrt{P}(e^{z/2} - 1)} = e^{z/2} - 1.$$

If $z < 0$, write $z = -a$ with $a > 0$. The informed trader sells the base asset into the pool and removes the quote asset. The base-asset input and quote-asset output are

$$x_{\text{in}} = \frac{L}{\sqrt{P}}(e^{a/2} - 1), \qquad y_{\text{out}} = L\sqrt{P}(1 - e^{-a/2}).$$

The stale-price loss in quote-asset terms is

$$\Pi_{\text{stale}} = y_{\text{out}} - P' x_{\text{in}} = L\sqrt{P}(1 - e^{-a/2})^2.$$

The hook charges the fee on the risky input, so the fee is valued as $P' f x_{\text{in}}$. Setting $P' f x_{\text{in}} = \Pi_{\text{stale}}$ gives $f = e^{a/2} - 1$. Therefore both directions reduce to $f^\star(z) = e^{|z|/2} - 1$. Locally,

$$f^\star(z) = \frac{|z|}{2} + O(z^2),$$

so the half-gap rule is the first-order approximation to exact stale-loss recovery over small oracle-pool gaps.

This is the same square-root premium that appears in the hook implementation. Written in oracle and pool mids, the exact premium is

$$\texttt{exactPremium} = \begin{cases} \sqrt{\texttt{oracle_mid}/\texttt{pool_mid}} - 1, & \text{oracle above pool,} \\ \sqrt{\texttt{pool_mid}/\texttt{oracle_mid}} - 1, & \text{oracle below pool.} \end{cases}$$

The deterministic hook fee is

$$\texttt{hookFee} = \texttt{baseFee} + \alpha \cdot \texttt{exactPremium},$$

subject to `maxFee`. The current Solidity hook still implements this path through `alphaBps`. The v4 grid in this paper does not sweep α; it varies the stale-gap trigger, base fee floor, auction concession path, and max-fee cap around the existing fee law.

The identity is an accounting target, not a command to maximize the swap fee. The auction layer determines how the stale-price value is split between LP recovery and the repricer. For each auction candidate, the simulator computes the scheduled concession, applies the LP fee floor, then defines realized solver payment as

$$\texttt{solver_payment} = \texttt{gross_lvr} - \texttt{lp_fee}.$$

Rows are reported from realized clears subject to the fee cap. Under this accounting, high base fees reduce clear rates rather than creating LP recapture above 100%.

6 Liquidity-Width Guard

A fee rule controls the price paid by informed trades against stale prices. It does not by itself control how much new liquidity is exposed to those trades. The implemented hook therefore also contains a width and centering guard for incoming LP positions. This guard is not the main empirical result in the October auction replay, but it completes the hook-level control system: `beforeSwap` prices toxic flow after a stale state is detected, while `beforeAddLiquidity` limits marginal-liquidity shape before stale flow arrives.

The guard exists because dynamic fees can recapture stale-loss value after an informed trade occurs, but they do not bound how much curvature risk a newly admitted LP range contributes

ex ante. Very narrow concentrated-liquidity ranges can multiply stale loss per dollar of LP value by tens or hundreds of times. Under the amplification factor derived below, a 10% centered span amplifies gross LVR by about 42.47×, while a 1% centered span amplifies it by about 402.50×. The width guard rejects ranges that violate the pool's volatility-latency LVR budget before those ranges become the marginal liquidity that toxic flow trades against.

For a Uniswap v3/v4-style centered range $[P_a, P_b]$, define total log width $W = \log(P_b/P_a)$ and center $P_0 = \sqrt{P_a P_b}$. At the center, the position value is

$$V(P_0) = 2L\sqrt{P_0}(1 - e^{-W/4}).$$

The local stale-loss numerator is the same constant-product numerator for the same liquidity parameter L, while the value denominator shrinks as the range narrows. The normalized gross-LVR rate at the center is therefore amplified to

$$\frac{\ell(\sigma, P_0)}{V(P_0)} = \frac{\sigma^2}{8(1 - e^{-W/4})}.$$

If the designer budgets gross LVR over an effective latency window Δ at no more than ϵ of position value, it suffices to require

$$\frac{\sigma^2 \Delta}{8(1 - e^{-W/4})} \leq \epsilon, \qquad W \geq W_{\min} = -4 \log\left(1 - \frac{\sigma^2 \Delta}{8\epsilon}\right).$$

The Solidity hook maps this formula into ticks. The add-liquidity hook first rejects positions whose midpoint is outside the oracle-centered tolerance, then calls the min-width helper. Internally, `_widthFactorWad` computes $1 - e^{-W/4}$ from the supplied width ticks through `TickMath`, and `_minWidthTicks` binary-searches the smallest spacing-aligned width satisfying the budget. The October auction replay does not claim to validate this guard empirically; it is retained as hook-level protection against very narrow ranges multiplying stale-loss per unit of position value.

7 Methodology

The backtest proceeds in three layers. First, every observed block in the October 2025 windows is paired with the latest Chainlink reference update available at or before the next block. The simulator compares the reference price to the pool price, computes the unsigned stale gap with `gap_bps()`, and constructs the direction-aware correction trade. Second, the shared auction trigger decides whether an auction can open:

$$\texttt{is_auction_eligible(state, trigger_gap_bps)} \iff \texttt{stale_gap_bps_before} \geq \texttt{trigger_gap_bps}.$$

Third, each stale opportunity is evaluated under a Dutch auction path. The starting concession is measured in bps of gross stale LVR. Concession grows per second until the solver clears, the fee cap binds, or the short auction expires.

The input data is stronger than a synthetic price path. The export bundle contains the pool snapshot, initialized ticks, liquidity events, observed swap samples, and oracle/reference updates needed for exact V3 replay and fee-curve checks. Those paths validate pool-state reconstruction and classify observed flow, while the auction grid isolates stale repricing by applying correction trades to the same October windows. What is still not modeled is competitive routing: whether

each historical user swap would have chosen this hooked pool, another onchain pool, a CEX, or no trade under the changed fee and pending-auction state.

To separate the execution question from the missing routing layer, this revision also runs a cached exact replay. That run reconstructs V3 pool state, classifies observed flow, replays historical swaps through the fixed, hook, linear, and log fee policies, and overlays the Dutch-auction branch on the same windows. It compares two auction rules on identical inputs: a broad rule that offers every stale repricing opportunity after a solver-cost reserve, and a narrower rule that starts only when the auction improves on the base hook after a base-hook reserve. It still does not model where a user swap would route under the changed hook state, and it does not feed later uninformed and informed swaps through a full pending-auction simulation.

The fixed-fee V3 baseline uses the same modeled informed repricing opportunities but charges only the pool's ordinary V3 fee tier:

$$\texttt{fixed_fee} = \texttt{informed_input_notional} \times \texttt{fixed_fee_bps}/10{,}000.$$

This baseline is deliberately conservative and local. It asks what the existing production fee tier recovers from the same stale-loss events, not what a different routing environment might have done.

The grid is discrete rather than optimized by a search harness. It evaluates:

Parameter	Values
`trigger_gap_bps`	5, 10, 25, 50
`base_fee_bps`	1, 5, 30
`start_concession_bps`	10, 30, 100
`concession_growth_bps_per_sec`	0.5, 1.0, 5.0
`max_fee_bps`	500, 2500, 5000

The test covers four pools, 324 parameter sets per pool, and 1,296 pool-parameter rows in `reports/sensitivity_grid_combined.csv`. The window-level output contains 40,176 rows in `reports/sensitivity_grid_windows.csv`. The dataset spans the October 2025 month slice, with oracle timestamps from 2025-09-30 23:55:23 UTC through 2025-10-31 23:57:23 UTC and block range 23,479,220 to 23,700,753. Each pool has 31 fixed windows of roughly one day, for 124 pool-windows total.

8 Results

The main result is that the auction model admits parameter sets that beat fixed-fee V3 on all four pools. The recommended parameter set is:

$$(10,\ 5,\ 10,\ 0.5,\ 2500),$$

in the order trigger gap, base fee, starting concession, concession growth, and max fee. The choice is not an unconstrained optimum. It is a tied-best accepted parameter set after requiring each pool to beat its fixed-fee V3 baseline and maintain clear rate above the working 0.5 acceptance threshold; within that tie, it uses the lower concession growth and 2500 bps max-fee cap.

The 2500 bps max fee is a stress cap, not the typical charged fee. Realized fees are determined by the stale gap, auction concession, fee floor, and clear condition.

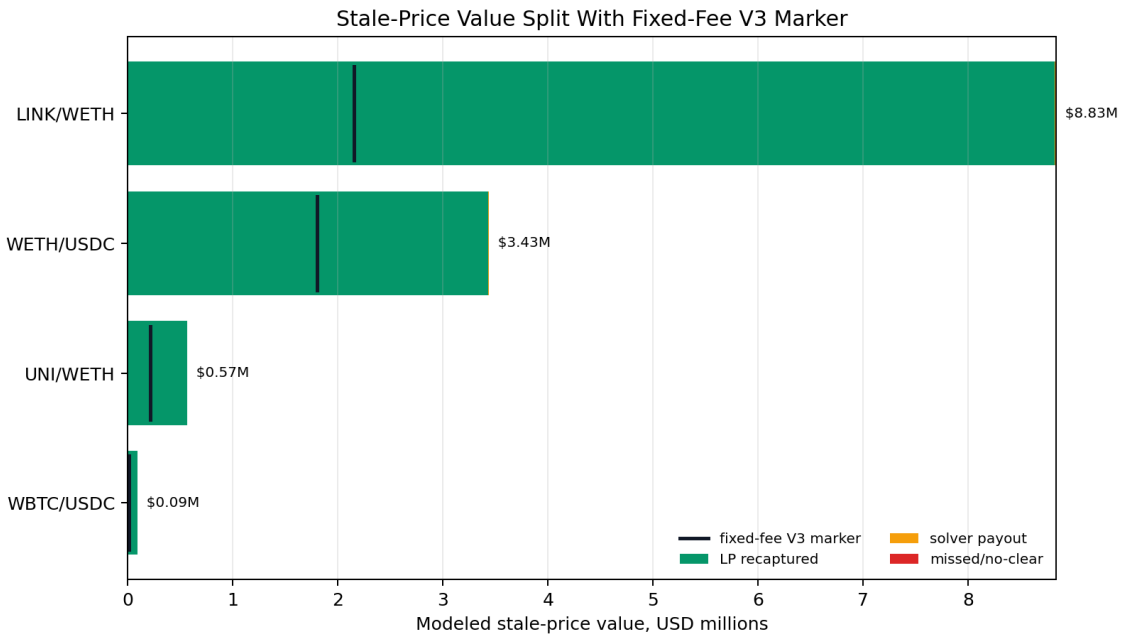

Figure 1: USD-equivalent stale-price value split for the selected parameter set by pool. Green bars show LP-recovered stale value, orange shows solver payout, red shows missed/no-clear value, and the black marker shows fixed-fee V3 recovery on the same modeled opportunities.

Pool	Recapture	Clear rate	V3 baseline	Triggers	Solver payout
WETH/USDC	99.9000%	1.0000	52.5709%	1242	10.0000 bps
WBTC/USDC	99.9000%	1.0000	16.9001%	800	9.9995 bps
LINK/WETH	99.9000%	1.0000	24.4175%	3163	10.0000 bps
UNI/WETH	99.9000%	1.0000	37.7356%	2209	10.0000 bps

The uniform 99.9% recapture is not a claim that the four pools naturally have identical loss profiles. It follows from the auction rule for the selected parameter set. The starting concession gives the solver 10 bps of gross stale-LVR value, not 10 bps of trade notional, so a quickly clearing auction leaves roughly 99.9% of gross LVR to LPs. The more informative cross-pool evidence is that the parameter set clears on all four pools and beats each pool's fixed-fee V3 baseline.

The table below converts the solver reward from bps into dollars. Using the same USD conversion as Figure 1, the selected parameter set pays solvers about $12.9k across 7,414 filled auctions in the October replay, or $1.74 per auction before gas.

Pool	Filled auctions	Stale value	Solver payout	Avg payout
LINK/WETH	3163	$8.83M	$8.8k	$2.79
WETH/USDC	1242	$3.43M	$3.4k	$2.77
UNI/WETH	2209	$565.1k	$565.12	$0.26
WBTC/USDC	800	$92.7k	$92.72	$0.12
Total	7414	$12.93M	$12.9k	$1.74

This is the main protocol-design caveat in the results. The average-payout column is also a break-even cost threshold: a solver must clear the auction for less than that amount, including

gas and operational overhead, before the opportunity is profitable. Lower-cost L2 deployment, for example Base or Arbitrum, therefore changes the interpretation materially. The LINK/WETH and WETH/USDC rows look more plausible if settlement costs are well below a few dollars per auction, while the UNI/WETH and WBTC/USDC rows still require batching, larger stale-price events, or more volume because their average payouts are measured in cents. The mechanism creates solver revenue, but this four-pool month does not by itself prove that live solvers would compete profitably.

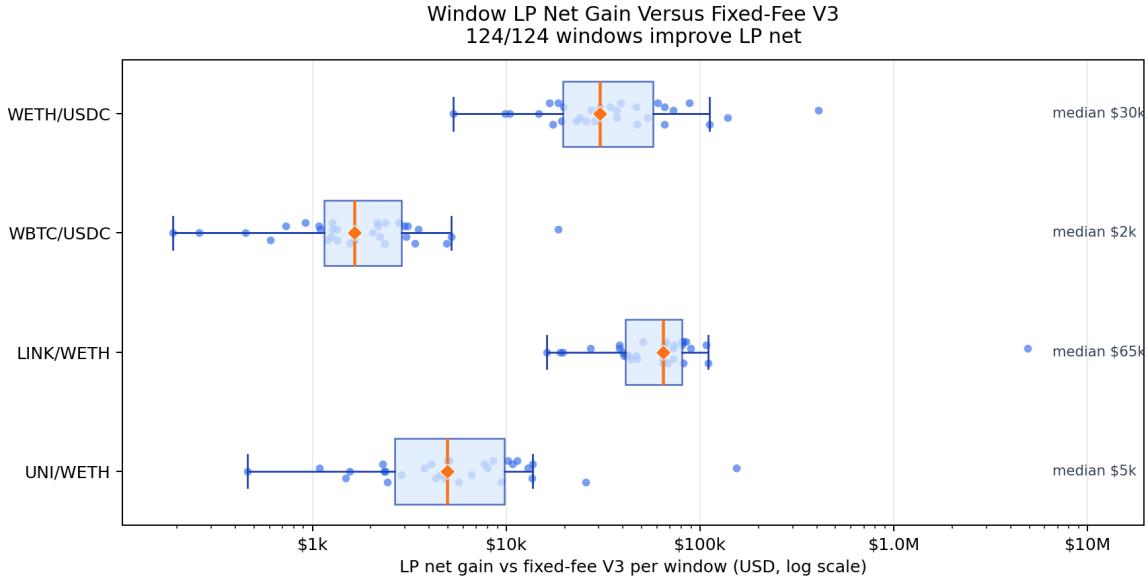

Figure 2: Window-level LP net gain for the recommended parameter set versus fixed-fee V3.

The baseline table explains the economic gap. Fixed-fee V3 collects a static fee on notional, while stale loss is measured against the stale repricing opportunity. The implied stale-loss bps column backs out the average modeled stale loss from the observed baseline recapture.

Pool	V3 fee	V3 recapture	Implied stale loss	Gain vs V3
WETH/USDC	30 bps	52.5709%	57.07 bps	47.3291 pp
WBTC/USDC	5 bps	16.9001%	29.59 bps	82.9999 pp
LINK/WETH	30 bps	24.4175%	122.86 bps	75.4825 pp
UNI/WETH	30 bps	37.7356%	79.50 bps	62.1644 pp

For LINK/WETH, a 30 bps fixed fee is measured against modeled stale-loss events averaging about 122.86 bps, so the production fee tier recovers only 24.4175% of the stale-price loss in this replay. For WBTC/USDC, the 5 bps fee tier recovers only 16.9001%. This is the clearest leakage result in the study: existing fixed-fee tiers leave a large share of modeled stale LVR unrecovered.

8.1 Observed-Flow Replay Check

The additional exact replay uses the cached multi-pool manifest from the May 4, 2026 run. It is not the same object as the correction-trade sensitivity grid above. Instead, it asks whether the historical replay machinery and auction overlay tell the same qualitative story when run on observed swap samples and informed/uninformed labels.

Coverage metric	Value
Replay windows	54
Window groups	12
Pools	7
Routine / historical stress windows	36 / 18
Observed prefix swap rows	7,106
Exact replay reliable windows	54 / 54
Fee formula checks passed	54 / 54
Maximum p99 exact-replay error	2.05×10^{-7}
Omitted cached sources	0

The hook-based auction rule is more selective. Its mean trigger rate is 0.9825% across windows, compared with 5.8233% for the broad all-stale rule. Every hook-based window with at least one auction trigger has a 1.0 fill rate, and the run records no oracle fail-closed events under that rule.

Regime	Windows	Positive delta	Old LP net	New LP net	Delta	95% CI for delta
Overall	54	28	-2.8646	2.7735	5.6381	[1.7364, 9.5666]
Normal	36	18	-2.2242	3.0039	5.2281	[0.5103, 10.5484]
Stress	18	10	-4.1454	2.3127	6.4581	[0.0000, 11.0710]

The LP net columns are reported in each window's native quote units, so they should not be read as a cross-pool P&L total. The clearest result is the sign: the hook-based rule improves LP net versus the broad all-stale rule in 28 windows, leaves 26 unchanged, and worsens no window. The unchanged windows are mostly cases with no auction opportunity under either rule, or windows where the base hook already captures the relevant stale-flow accounting.

The sensitivity table replaces the earlier correlation-matrix framing. The largest one-step effect is `base_fee_bps`; increasing the base fee lowers measured recapture because it removes marginal clears. The next-largest effects are the starting concession and trigger gap. `max_fee_bps` and concession growth are second-order over this October slice.

Parameter step	Mean delta	Std delta	Pools improved
`base_fee_bps` up	-0.986049 pp	0.562620 pp	0/4
`start_concession_bps` up	-0.699985 pp	0.000013 pp	0/4
`trigger_gap_bps` up	-0.576773 pp	0.424254 pp	0/4
`start_concession_bps` down	0.199997 pp	0.000004 pp	4/4

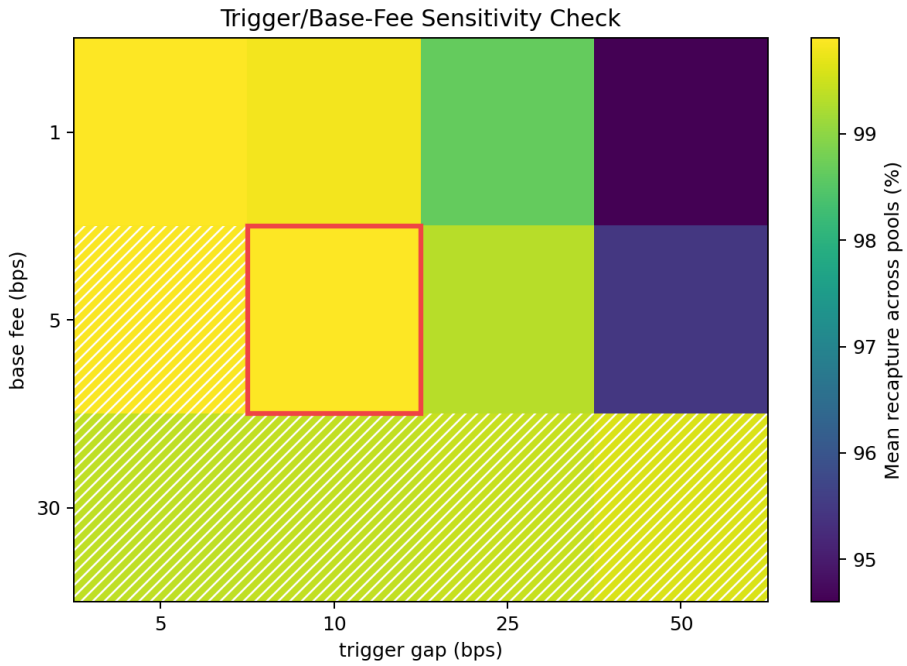

Figure 3: Trigger-gap by base-fee sensitivity check. The other auction parameters are fixed at the recommended values; hatched entries have at least one pool below 0.9 clear rate, and the red outline marks the recommended parameter set. The 0.5 clear-rate threshold is the coarse acceptance filter; the 0.9 hatching is a stricter visual robustness diagnostic.

9 Discussion

The evidence supports a specific story. The hook fee formula supplies the stale-loss accounting target. The stale-gap trigger supplies a scale-free rule for opening auctions. The Dutch auction supplies an execution path. With those pieces combined, the October 2025 replay finds a small set of parameter sets that outperform fixed-fee V3 across all four tested pools.

The change from a 5 bps trigger to a 10 bps trigger follows from the cross-pool consistency check. At 5 bps, the simulator opens auctions on more marginal stale states. Those weak opportunities still beat fixed-fee V3, but they clear less reliably and vary more across pools. At 10 bps, the parameter set is more selective: it produces 99.9000% mean recapture and 1.0 clear rate on all four pools.

This does not mean the production design should mechanically target 99.9% LP recapture. It means the tested auction layer can leave a visible solver reward while returning nearly all modeled stale LVR to LPs. In production, gas costs, solver competition, reference disagreement, and routing responses would move the recommended concession. The paper's claim is therefore split deliberately: the fee formula is the accounting foundation; the October grid is historical replay evidence; and production deployment still needs solver-market validation.

Solver incentive design is not limited to the explicit concession. First, batched correction can let one solver correct many stale pools inside one PoolManager unlock, amortizing fixed gas without increasing the per-pool concession paid by LPs. Second, a small reserve funded from ordinary-flow fees could pay for correction reliability, provided the diverted fee is smaller than the stale-price losses avoided and does not push ordinary flow away. A more speculative design is to let the winning solver settle user swaps against its own inventory while the hook still enforces the AMM-plus-hook price as the execution benchmark and falls back to the AMM if the solver fails to fill. That could let solvers earn inventory spread rather than only the explicit concession, but it expands the security and liveness surface and should be treated as future design work rather than an assumption in this study.

We recommend the 10 bps trigger gap, 5 bps base fee, 10 bps starting concession, 0.5 bps/sec growth, and 2500 bps max fee as the current deployment candidate because it outperforms the fixed-fee V3 baseline on all four pools, preserves execution in the replay, and leaves a visible solver payout. We do not recommend treating parameter sets above 100% recapture as evidence of superior LVR recovery; in production, those settings risk auctions that do not clear.

10 Limitations

The study covers one month, with the headline grid focused on four pools and the observed-flow replay spanning seven pools. It omits realistic gas costs and competitive solver dynamics. The paper's headline auction grid tests stale-price correction trades rather than full competitive routing. The observed-flow replay adds historical swaps through fee curves and a Dutch-auction overlay, but it still does not decide whether each historical user swap would route to this pool, another V3 pool, a V4 hook pool, a CEX, or no venue under the changed hook state. It also does not run all later uninformed and informed swaps through a full pending-auction simulation. Chainlink is the primary reference oracle, with Pyth, Binance, and deep-pool references reserved for sensitivity checks. The grid does not sweep `alphaBps`; alpha calibration remains separate from the public auction-layer sensitivity table. The working 0.5 clear-rate acceptance threshold is also a methodology choice rather than a settled protocol requirement. The liquidity-width guard is derived and implemented in the hook, but the October auction grid does not test LP repositioning

behavior, off-center range demand, or range-width effects on realized order flow.

11 Future Work

The next empirical step is a multi-month replay with the same stale-gap trigger and parameter grid. The next market-design step is a two-pool competitive-routing simulation: two pools with the same liquidity distribution and different fee policies, evaluated under simulated trader flow. The auction model should also be extended to include realistic gas costs, solver profit margins, multiple solvers, reference-oracle disagreement, and explicit fallback behavior when no solver clears.

12 Methodology Notes

- The clear-rate threshold for accepting a parameter set is 0.5 in this study; Figure 3 uses 0.9 only as a stricter visual robustness diagnostic.
- One-step sensitivity changes compare one grid value away from a lower-median baseline where needed.
- Brownian-motion synthetic paths are treated as engineering validation rather than headline empirical evidence.
- `alphaBps` calibration remains separate from the public auction-layer grid.

13 Conclusion

OracleAnchoredLVRHook should be read as a fee formula plus execution-design proposal. The fee formula defines what the hook should recover from informed trades against stale prices. The stale-gap trigger defines when a pool is stale enough to act. The Dutch auction then determines how repricing happens. On the October 2025 four-pool replay, the recommended parameter set recaptures about 99.9% of modeled stale LVR, compared with fixed-fee V3 recapture between 16.9001% and 52.5709%. We propose this parameter set because it converts the paper's core thesis into a testable deployment hypothesis: pool-oracle stale gaps should trigger repricing auctions when the design can beat fixed-fee V3 and maintain execution.

14 Appendix Check

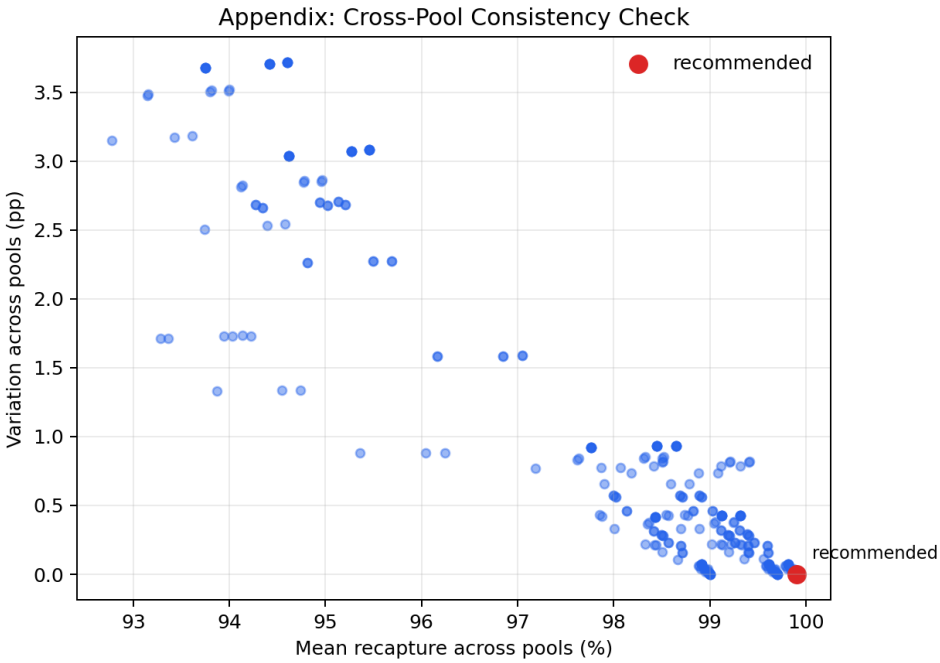

Figure 4: Appendix check: cross-pool mean recapture versus variation across pools for every parameter set.

References

1. Jason Milionis, Ciamac C. Moallemi, Tim Roughgarden, and Anthony Lee Zhang. "Automated Market Making and Loss-Versus-Rebalancing." arXiv:2208.06046, August 2022, revised May 2024. DOI: 10.48550/arXiv.2208.06046.

2. Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. "Uniswap v3 Core." Uniswap Labs whitepaper, March 2021. uniswap.org/whitepaper-v3.pdf.

3. Hayden Adams, Moody Salem, Noah Zinsmeister, Sara Reynolds, Austin Adams, Will Pote, Mark Toda, Alice Henshaw, Emily Williams, and Dan Robinson. "Uniswap v4 Core." Uniswap Labs whitepaper, August 2024. app.uniswap.org/whitepaper-v4.pdf.

4. Austin Adams, Ciamac C. Moallemi, Sara Reynolds, and Dan Robinson. "am-AMM: An Auction-Managed Automated Market Maker." arXiv:2403.03367, March 2024.

5. Eric Budish, Peter Cramton, and John Shim. "The High-Frequency Trading Arms Race: Frequent Batch Auctions as a Market Design Response." *The Quarterly Journal of Economics*, 130(4):1547–1621, 2015. DOI: 10.1093/qje/qjv027.